



Department of Chemical Engineering
College of Engineering
Bicol University
Legazpi City, Albay
2nd Semester, A/Y 2025-2026



COMPUTER FUNDAMENTALS AND PROGRAMMING (BES 10a)

PORTFOLIO 3 FUNCTIONS

ARAÑA, JANELE A.
BSChE-1A
STUDENT

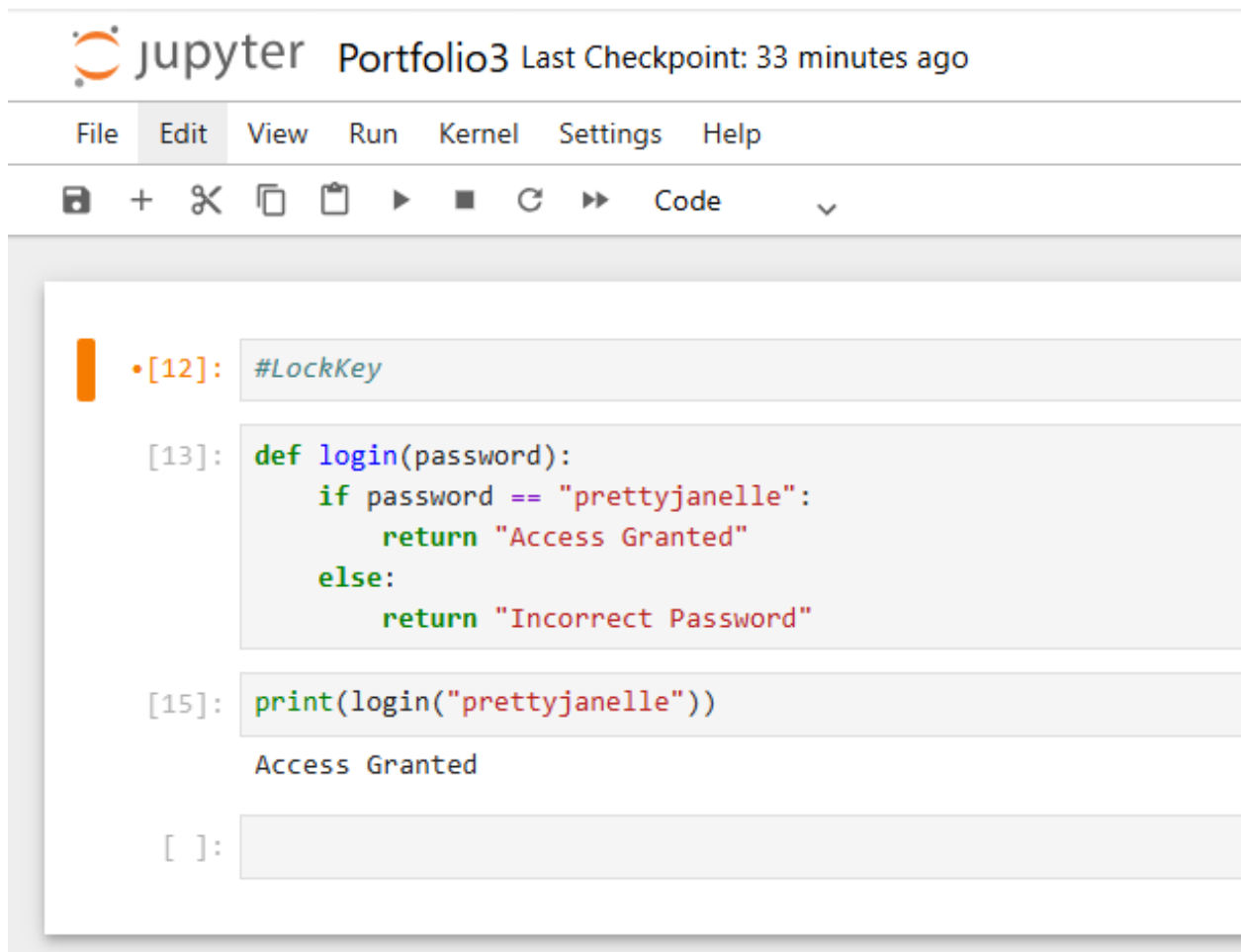
ENGR. VINCENT CALIMLIM
PROFESSOR

Introduction

In Portfolio 3, I chose a daily life scenario that focuses on the concept of functions, lesson 4 in Computer Fundamentals and Programming. This involves using passkey or password system in gadgets such as smartphones, laptops. Tablets, and digital accounts.

In everyday life, passkey systems are important for protecting personal information and ensuring security. Through functions, the program can repeatedly verify user inputs, display messages, and grant or deny access efficiently. This demonstrates how functions can simplify repetitive tasks and make programs more organized and practical. I called this program "LockKey."

Design of the Program



The screenshot shows a Jupyter Notebook interface with the title "Portfolio3" and a timestamp "Last Checkpoint: 33 minutes ago". The interface includes a menu bar with "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". Below the menu bar is a toolbar with icons for saving, adding, deleting, copying, pasting, running, and other actions. The main area displays three code cells. The first cell is a comment "#LockKey". The second cell defines a function "login" that takes a "password" argument and returns "Access Granted" if the password is "prettyjanelle", or "Incorrect Password" otherwise. The third cell calls the "login" function with the argument "prettyjanelle", and the output "Access Granted" is displayed below the code.

```
[12]: #LockKey

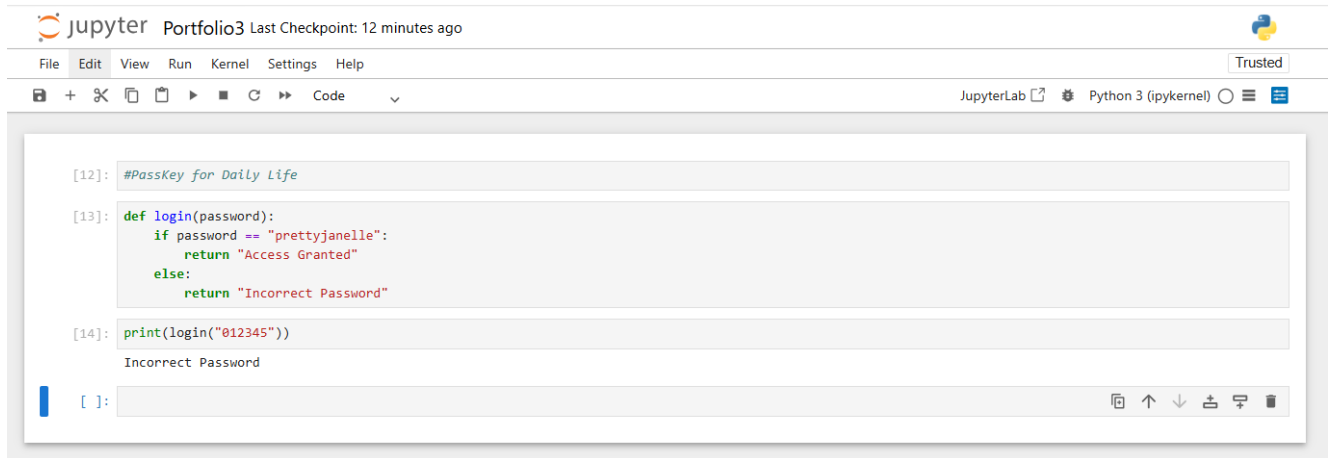
[13]: def login(password):
        if password == "prettyjanelle":
            return "Access Granted"
        else:
            return "Incorrect Password"

[15]: print(login("prettyjanelle"))
        Access Granted

[ ]:
```

Figure 1.

The function denied access and showed an error message because the user submitted the wrong password. This demonstrates how features in devices and digital systems can enhance security and verify user input.



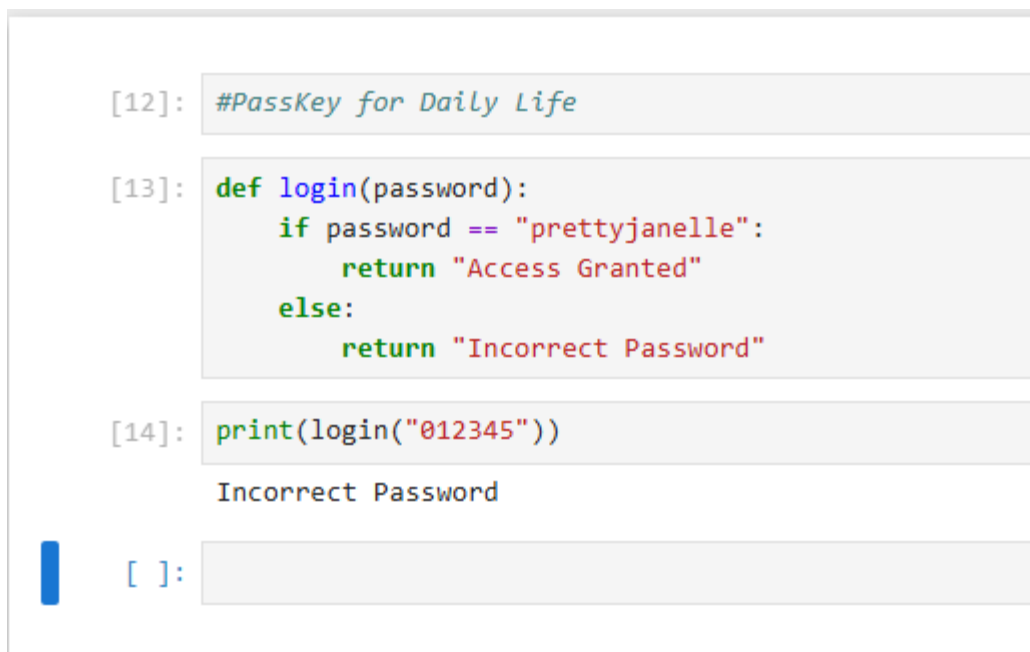
The screenshot shows a JupyterLab window titled "Portfolio3" with a last checkpoint of 12 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The code editor contains three cells:

```
[12]: #PassKey for Daily Life
```

```
[13]: def login(password):  
      if password == "prettyjanelle":  
          return "Access Granted"  
      else:  
          return "Incorrect Password"
```

```
[14]: print(login("012345"))  
      Incorrect Password
```

The output of the third cell is "Incorrect Password". The interface also shows "JupyterLab" and "Python 3 (ipykernel)" in the top right corner.



This block provides a zoomed-in view of the code cells from the JupyterLab interface:

```
[12]: #PassKey for Daily Life
```

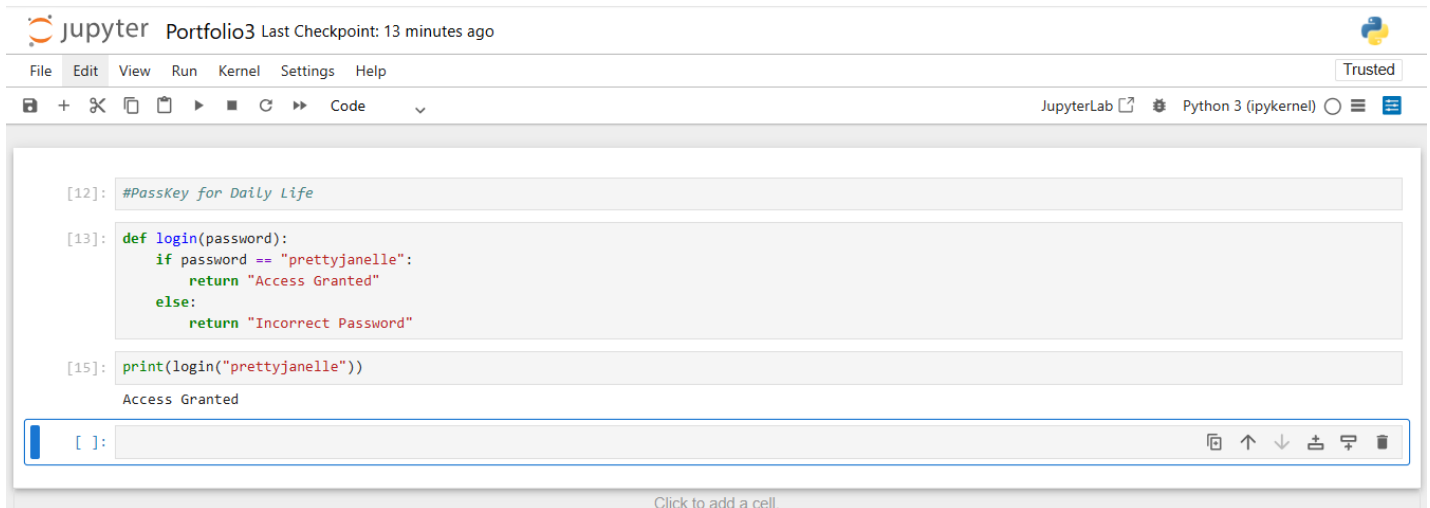
```
[13]: def login(password):  
      if password == "prettyjanelle":  
          return "Access Granted"  
      else:  
          return "Incorrect Password"
```

```
[14]: print(login("012345"))  
      Incorrect Password
```

The code is displayed in a clean, monospaced font with syntax highlighting.

Figure 2.

The function successfully allowed access to the system or device when the user submitted the proper password. This illustrates how functions can automate authentication procedures in real-world applications.



The screenshot shows a JupyterLab window titled "Portfolio3" with a "Last Checkpoint: 13 minutes ago" status. The interface includes a top menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main area contains three code cells. The first cell (index 12) has a comment: `#PassKey for Daily Life`. The second cell (index 13) defines a `login` function that checks if the password is "prettyjanelle"; if so, it returns "Access Granted", otherwise it returns "Incorrect Password". The third cell (index 15) calls `print(login("prettyjanelle"))`. Below the code cells, the output "Access Granted" is displayed. At the bottom, there is an input field for a new code cell and a "Click to add a cell." prompt.

```
[12]: #PassKey for Daily Life

[13]: def login(password):
      if password == "prettyjanelle":
          return "Access Granted"
      else:
          return "Incorrect Password"

[15]: print(login("prettyjanelle"))

Access Granted
```



This block provides a closer look at the code cells from the previous screenshot. It shows the same three cells: a comment, a function definition, and a function call. The output "Access Granted" is visible below the function call cell. The input field at the bottom is also visible.

```
[12]: #PassKey for Daily Life

[13]: def login(password):
      if password == "prettyjanelle":
          return "Access Granted"
      else:
          return "Incorrect Password"

[15]: print(login("prettyjanelle"))

Access Granted
```

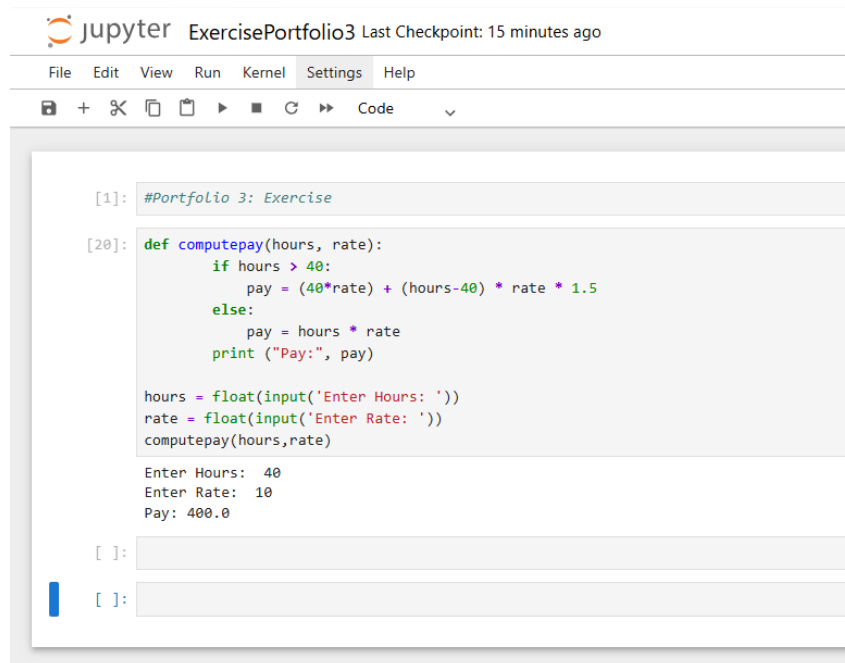
Exercises

Rewrite your pay computation with time-and-a-half for overtime and create a function called `compute pay` which takes two parameters (`hours` and `rate`).

Enter Hours: 45

Enter Rate: 10

Pay: 475.0



The screenshot shows a Jupyter Notebook window titled "ExercisePortfolio3" with a last checkpoint of 15 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for saving, adding, deleting, and running code. The code cell [20] contains a function definition for `compute pay` that calculates overtime pay at 1.5 times the rate. Below the code, the input and output of the function are shown: "Enter Hours: 40", "Enter Rate: 10", and "Pay: 400.0".

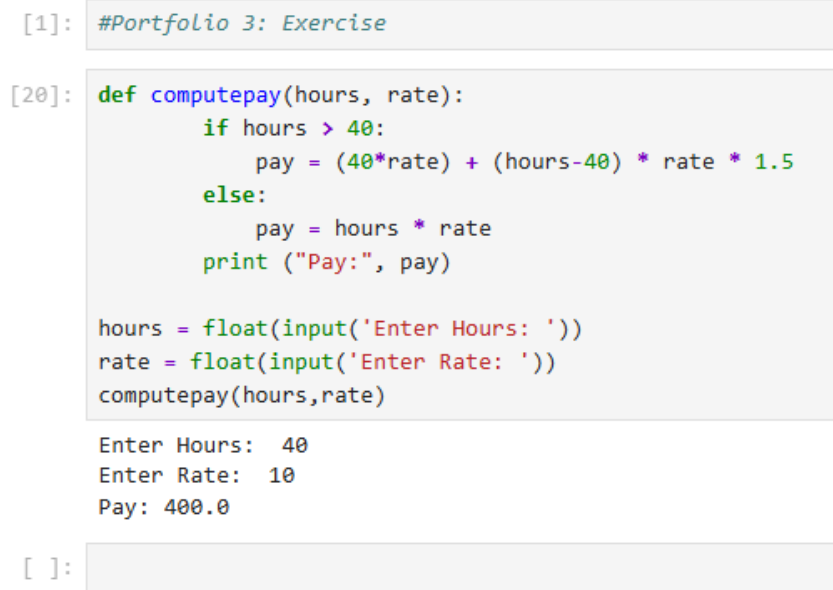
```
[1]: #Portfolio 3: Exercise

[20]: def compute pay(hours, rate):
        if hours > 40:
            pay = (40*rate) + (hours-40) * rate * 1.5
        else:
            pay = hours * rate
        print ("Pay:", pay)

        hours = float(input('Enter Hours: '))
        rate = float(input('Enter Rate: '))
        compute pay(hours,rate)

        Enter Hours: 40
        Enter Rate: 10
        Pay: 400.0

[ ]: 
[ ]:
```



This screenshot shows the same Jupyter Notebook interface as the previous one, but with the input values changed to 45 hours and 10 rate. The function definition remains the same, and the output shows the correct overtime pay calculation: "Enter Hours: 45", "Enter Rate: 10", and "Pay: 475.0".

```
[1]: #Portfolio 3: Exercise

[20]: def compute pay(hours, rate):
        if hours > 40:
            pay = (40*rate) + (hours-40) * rate * 1.5
        else:
            pay = hours * rate
        print ("Pay:", pay)

        hours = float(input('Enter Hours: '))
        rate = float(input('Enter Rate: '))
        compute pay(hours,rate)

        Enter Hours: 45
        Enter Rate: 10
        Pay: 475.0

[ ]: 
```